

다중 맵 — 사이클 분해 + 테스트 케이스

날짜: 2026-05-03 선행 spec: 다중 맵 — 자료구조 설계.pdf 적용 방법론: 코딩 과업 관리 실천 방법론

§0. 방법론 적용 결

0-1. 인용 방법론

- 크게 쪼개고
- 쪼갠 각각을 테스트로 검증 가능한 기능 단위까지 다시 쪼갬 후
- 3~7개 단위로 투두에 올려서 치우고 체크하라

사이클 = ① 구현 단위 정의 → ② 구현 → ③ 디버깅 장치 통과 → ④ 테스트 통과 → ⑤ 체크 & 다음

쪼개기 기준 = 시간 X, 구조의 완성 O. "이 단위가 테스트로 검증 가능한가" 가 충분히 작은지 판단 기준.

0-2. 본 차 적용

다중 맵 도입 = 큰 과업.

- 1단계 (크게): M-1 ~ M-4 (4 큰 단위)
- 2단계 (기능 단위): 각 M 을 testable unit 으로 다시 쪼갬 (총 ~ 22 unit)
- 3단계 (3~7 개 묶음): 각 사이클에 5~7 개 unit 넣어 체크

각 unit 에는: - 구현 단위 정의 — 무엇을 만들 것인가 (한 줄) - 완료 조건 — 테스트 결과 명시 (자동 검증 가능 결과) - 디버깅 장치 — 사이클 안 검증 결과 (push_warning / 콘솔 / 헤드리스) - 통과 기준 — 모든 완료 조건 PASS

§1. 큰 단위 (M-1 ~ M-4)

M	구현 단위 정의	완료 조건
M-1	1 맵 결의 schema 만 다중 맵 결로 wrapping. 외부 동작 0 변경.	회귀 186 PASS + 신규 schema 검증 PASS
M-2	두번째 맵 (ruin_01) + transition 슬롯 양방향.	시뮬: 전이 → 도착 → 복귀 → 회귀 시 시작 맵 복귀
M-3	UI 다중 그리드. current_map_id 변경 시 그리드 토글.	시각: 맵 A / 맵 B 스크린샷, 마커 + HUD 정확

M	구현 단위 정의	완료 조건
M-4	세이브 마이그레이션 + 다중 맵 직렬화.	세이브/로드 round-trip: 맵 B 에서 저장 → 로드 시 맵 B 복원

§2. M-1 — Schema Wrapping (1 맵 유지)

2-1. 사이클 1 — 자료구조 (5 unit)

#	unit	완료 조건
1.1	MAPS const 신설, 기존 WORLD_TERRAIN/SPAWN_POS/TILE_ENCOUNTERS 를 MAPS["field_01"] 안으로 이전	GameData.MAPS.has("field_01") + MAPS["field_01"]["terrain"] 16열×12행 + MAPS["field_01"]["spawn"] == Vector2i(1,6) + MAPS["field_01"]["encounters"].size() == 5
1.2	STARTING_MAP const 신설	GameData.STARTING_MAP == "field_01"
1.3	기존 const (WORLD_TERRAIN / TILE_ENCOUNTERS / SPAWN_POS) 폐기	grep 결과 코드베이스 어디에도 미참조
1.4	TERRAIN_RULES 는 전역 그대로 유지	GameData.TERRAIN_RULES 변경 X
1.5	자동 검증 스크립트 (test_multi_map_schema.gd) — 위 4 항목 검사	--headless 실행 PASS

2-2. 사이클 2 — run_data 필드 (5 unit)

#	unit	완료 조건
2.1	_start_internal_run 가 current_map_id / player_pos 신설	RunManager.run_data["current_map_id"] == "field_01" 회차 시작 시
2.2	visited_by_map / explored_by_map 신설	run_data["visited_by_map"]["field_01"][SPAWN] == true , run_data["explored_by_map"]["field_01"] == {}
2.3	기존 visited_tiles / explored_tiles 폐기	run_data 에 두 키 없음
2.4	seen_this_run 그대로 유지 (글로벌)	run_data["seen_this_run"] == {} 회차 시작

#	unit	완료 조건
2.5	회귀 후 모든 필드 자동 리셋 검증	<code>end_internal_run</code> 후 <code>current_map_id == STARTING_MAP</code> , <code>visited_by_map[STARTING_MAP][SPAWN] == true</code> 외 키 빔

2-3. 사이클 3 — 조회자 함수 (4 unit)

#	unit	완료 조건
3.1	<code>_current_map()</code> helper 신설	<code>RunManager._current_map()</code> 가 <code>MAPS["field_01"]</code> dict 반환
3.2	<code>_get_terrain_at</code> 가 <code>_current_map()</code> 경로 분기	<code>_get_terrain_at(Vector2i(1,6)) == "G"</code> , <code>_get_terrain_at(Vector2i(0,0)) == "F"</code>
3.3	<code>get_tile_encounter</code> 가 <code>_current_map()</code> 경로 분기	<code>get_tile_encounter(Vector2i(3,6))</code> 가 <code>regression_speech</code> 슬롯 반환
3.4	<code>_is_passable</code> / <code>get_terrain_name</code> 변경 없이 작동	(3.2 위에 얹혀있음 — 회귀 자동)

2-4. 사이클 4 — 액션 함수 (5 unit)

#	unit	완료 조건
4.1	<code>try_move</code> 의 <code>visited_tiles</code> 참조 → <code>visited_by_map[current_map_id]</code>	동 1칸 이동 후 <code>visited_by_map["field_01"][Vector2i(2,6)] == true</code>
4.2	<code>try_explore</code> 의 <code>explored_tiles</code> 참조 → <code>explored_by_map[current_map_id]</code>	탐험 후 <code>explored_by_map["field_01"][pos] == true</code>
4.3	<code>_check_on_enter</code> 분기 변경 없음 (조회자만 변경)	<code>regression_speech</code> 발화 그대로
4.4	<code>_check_on_explore</code> 분기 변경 없음	<code>found_letter</code> 발화 그대로
4.5	회귀 6 PASS	<code>test_event_1a/1a2/1b/2/grid/save_load</code> 모두 PASS

2-5. 사이클 5 — UI 호환 + 회귀 누적 (3 unit)

#	unit	완료 조건
5.1	<code>run_ui.gd</code> 의 <code>_build_grid_display</code> / <code>_refresh_grid_display</code> 가 <code>_current_map()</code> 경로 동작	시각: 그리드 / 마커 / 플레이어 정상 표시

#	unit	완료 조건
5.2	<code>_terrain_label</code> 가 맵 이름 표시	"평원 — 풀밭 (1, 6)" 경로 표시
5.3	신규 검증 <code>test_multi_map_schema.gd</code> 통과 + 회귀 6 PASS	7 / 7 PASS

2-6. M-1 통과 기준

- 회귀 186 + 신규 ~10 = 196 PASS
- 외부 동작 변경 0 (1 맵 결 그대로)
- 코드 어디에도 `WORLD_TERRAIN` / `SPAWN_POS` 직접 참조 없음

§3. M-2 — 두번째 맵 + Transition

3-1. 사이클 1 — `ruin_01` 정의 (3 unit)

#	unit	완료 조건
6.1	<code>MAPS["ruin_01"]</code> 신설 (10×8 또는 16×12 결, 단순 시작)	<code>MAPS["ruin_01"]["terrain"]</code> 정합 + <code>spawn Vector2i</code> + <code>encounters</code> 5~6 개 슬롯
6.2	<code>ruin_01</code> 안 슬롯 5~6 개 박기 (event 단편 + combat 1 + transition 복귀)	슬롯 dict 검증
6.3	자동 검증 — <code>MAPS</code> 의 모든 맵 키 / terrain / spawn / encounters 정합	<code>test_multi_map_schema.gd</code> 확장 PASS

3-2. 사이클 2 — Transition 슬롯 양방향 (4 unit)

#	unit	완료 조건
7.1	<code>field_01</code> 의 (15, 6) 에 transition 슬롯 (<code>target_map: "ruin_01", target_pos: (0, 5)</code>)	<code>MAPS["field_01"]["encounters"]</code> <code>[Vector2i(15,6)]["on_enter"]["kind"] == "transition"</code>
7.2	<code>ruin_01</code> 의 (0, 5) 에 transition 슬롯 (<code>target_map: "field_01", target_pos: (15, 6)</code>)	(7.1 의 역)
7.3	<code>_check_on_enter</code> 의 <code>"transition"</code> 분기 구현	슬롯 진입 → <code>current_map_id</code> 변경 + <code>player_pos = target_pos + visited_by_map[target_map]</code> 자동 박힘

#	unit	완료 조건
7.4	도착지 슬롯 자동 평가 (도착 칸에 또 슬롯 있으면 발화)	(0, 5) 에 도착하자마자 ruin 첫 슬롯 발화 흐름 검증

3-3. 사이클 3 — 시물 회귀 (3 unit)

#	unit	완료 조건
8.1	평원 (1, 6) → (15, 6) 이동 시물 → ruin_01 (0, 5) 도착 검증	<code>current_map_id == "ruin_01"</code> , <code>player_pos == Vector2i(0, 5)</code>
8.2	ruin_01 의 슬롯 발화 (event / combat) 정상 작동	regression_speech 결의 자료 결 검증 (개별 슬롯 발화 → 종료 → phase=map 복귀)
8.3	회귀 (<code>end_internal_run</code>) 후 <code>current_map_id == STARTING_MAP</code> 자동 복귀	<code>player_pos == MAPS[STARTING_MAP]["spawn"]</code> , <code>visited_by_map[ruin_01]</code> 빔

3-4. M-2 통과 기준

- 신규 검증 `test_multi_map_transition.gd` PASS
- 기존 회귀 PASS 유지
- 누적 검증 ~ 200 PASS

§4. M-3 — UI 다중 그리드

4-1. 사이클 1 — 그리드 미리 빌드 (4 unit)

#	unit	완료 조건
9.1	<code>_grid_roots: Dictionary</code> (map_id → Control) 신설	<code>run_ui</code> 의 <code>_grid_roots</code> 가 모든 맵의 Control 보유
9.2	<code>_build_grid_display</code> 가 MAPS 모든 맵의 그리드 미리 빌드	빌드 후 <code>_grid_roots.size() == MAPS.size()</code>
9.3	<code>_refresh_grid_display</code> 가 <code>current_map_id</code> 결로 가시성 토글	평원 visible / ruin invisible (또는 반대)
9.4	마커 (조우 / 플레이어 / visited) 결도 맵별	시각 검증 컷 1 (평원) + 컷 2 (ruin)

4-2. 사이클 2 — HUD (3 unit)

#	unit	완료 조건
10.1	<code>_terrain_label</code> 에 맵 이름 prefix	"평원 — 풀밭 (1, 6)" / "폐허 — 길 (3, 4)"
10.2	전이 슬롯 마커 신규 색 (예: 흰색 또는 개념적 색)	시각 컷 결 식별 가능
10.3	시각 검증 스크린샷 4 컷 (평원 + 평원 마커 / 폐허 + 폐허 마커)	사용자 육안 확인

4-3. M-3 통과 기준

- 시각 4 컷 (평원 / 폐허 / 전이 직후 / 회귀 직후)
- 회귀 PASS 유지

§5. M-4 — 세이브 마이그레이션

5-1. 사이클 1 — 신규 세이브 결 검증 (3 unit)

#	unit	완료 조건
11.1	신규 결 세이브 직렬화 검증 (<code>current_map_id</code> / <code>visited_by_map</code>)	<code>var_to_str</code> round-trip 일치
11.2	폐허에서 세이브 → 로드 → <code>current_map_id == "ruin_01"</code> , <code>player_pos</code> 일치	시물 검증
11.3	다른 맵 visited 도 보존	<code>visited_by_map[field_01]</code> + <code>visited_by_map[ruin_01]</code> 둘 다 일치

5-2. 사이클 2 — 옛 세이브 마이그레이션 (3 unit)

#	unit	완료 조건
12.1	<code>_migrate_save</code> 결 신설 (단일 맵 세이브 → 다중 맵 결)	옛 결 (<code>visited_tiles</code> 키) → 신 결 (<code>visited_by_map["field_01"]</code>) 자동 변환
12.2	옛 결 세이브 파일 1 개를 <code>test/fixtures/</code> 에 박고 로드 검증	로드 후 모든 필드 신 결로 정합
12.3	누락 필드 (<code>current_map_id</code> 없음) 결 → <code>STARTING_MAP</code> 자동 박힘	마이그레이션 결 검증

5-3. M-4 통과 기준

- `test_save_load.gd` 갱신 + 신규 검증 PASS
- 누적 검증 ~ 220 PASS

§6. 사이클 묶음 진행 표

각 사이클 = 3~7 unit. 한 사이클이 한 작업 단위.

단계	사이클 #	unit 수	누적 unit
M-1	1, 2, 3, 4, 5	5+5+4+5+3	22
M-2	1, 2, 3	3+4+3	32
M-3	1, 2	4+3	39
M-4	1, 2	3+3	45

총 12 사이클 / 45 unit.

§7. 사이클 안 검증 흐름 (방법론 ② → ④)

7-1. 매 unit 작업 흐름

1. 구현 단위 정의 (위 §2~5 의 표) 읽기
2. 완료 조건 명시 받기 (이미 표에 박힘)
3. 구현 — 코드 작성
4. 디버깅 장치 통과: - GDScript `push_warning` / `assert` - 콘솔 출력 (LimboConsole) - 헤드리스 검증 스크립트 PASS
5. 테스트 통과 — 사전 정의 완료 조건 모두 PASS
6. 체크 — TaskUpdate 결과 완료 표시 → 다음 unit

7-2. 사이클 안 단위 우선 순위

같은 사이클 안 unit 들은 자료구조 → 함수 → UI 순으로 진행 (자료구조 상향식 결).

7-3. 사이클 종료 시점

사이클의 모든 unit PASS + 회귀 6 PASS 유지 → 다음 사이클 진입.

회귀가 깨지면 그 자리에서 정지. 다음 진행 X.

§8. 신규 헤드리스 검증 스크립트 명세

8-1. `test_multi_map_schema.gd` (신규)

목적: M-1 자료구조 검증.

검증 항목 (~12): - `GameData.MAPS` 가 dict 이고 `STARTING_MAP` 키 보유 - 각 맵의 `terrain` / `spawn` / `encounters` 정합 - `STARTING_MAP` 이 `MAPS` 의 키 - `TERRAIN_RULES` 변경 없음 - `WORLD_TERRAIN` / `SPAWN_POS` / `TILE_ENCOUNTERS` 가 `GameData` 에 없음 (옛 결 제거 검증) - 회차 시작 시 `run_data` 의 신규 필드 모두 존재 + 정합값 - 회귀 후 `current_map_id == STARTING_MAP` , `visited_by_map` / `explored_by_map` 비움

8-2. `test_multi_map_transition.gd` (신규)

목적: M-2 전이 검증.

검증 항목 (~10): - `MAPS["ruin_01"]` 정합 - transition 슬롯 양방향 정합 (`target_map` / `target_pos` 가 다른 쪽 슬롯과 일치) - 평원 (15, 6) 진입 → ruin_01 (0, 5) 도착 - ruin_01 도착 시 자동 visited 박힘 - ruin_01 의 슬롯 발화 정상 - 복귀 (ruin (0, 5) → 평원 (15, 6)) 정상 - 회귀 후 시작 맵 복귀

8-3. `test_save_load.gd` (갱신)

기존 36 + 신규 ~8: - `current_map_id` 직렬화 / 복원 - `visited_by_map` 직렬화 / 복원 - 옛 결 세이브 마이그레이션 자동 변환 - 페허에서 세이브 → 로드 → 복원

8-4. `test_grid.gd` (갱신)

기존 49 검증 결로 신규 schema 호환 확인. 추가 신규 검증 ~3.

§9. 디버깅 장치 (방법론 ③)

9-1. 프레임워크 레벨 검증

- `assert(...)` — 자료구조 정합 검증 (예: `assert(MAPS.has(STARTING_MAP))`)
- `push_warning(...)` — 비정상 결 알림 (예: 알 수 없는 `target_map` / 슬롯 결 어긋남)
- `LimboConsole` — 런타임 디버그 (이미 박혀있음 — `show_run` / `move` / `explore` / `rest`)
- 헤드리스 자동 검증 — 매 사이클 종료 시 PASS 검증 (= 방법론 ④)

9-2. 신규 콘솔 명령 후보

- `move_to_map <map_id>` — 디버그 결 즉시 이동 (개발 결, 본 차로는 보류 가능)
- `show_map` — 현 맵 / visited / explored 덤프

§10. 진행 의지 / 미해결

10-1. 사이클 단위 진행 결

각 사이클 종료 시 사용자 합의 한 줄 (체크 OK / 다음 진입). 또는 M-1 ~ M-4 일괄 진행 후 PR 묶음.

10-2. 미해결

- (a) M-1 의 사이클 5 (UI 호환) 까지 한번에 갈지 / 사이클 1~4 만 진행 후 UI 별도 차로 갈지?
- (b) ruin_01 의 크기 (10×8 / 16×12 / 다른 결) — 본 차에는 콘텐츠 비중 0, schema 검증용이라 작은 결 충분
- (c) 사이클 묶음 (12 사이클) 을 별도 커밋으로 갈지 / M 단위 (4 커밋) 로 묶을지
- (d) 후속 연구 — "큰 기능에서 작은 기능 쪼개기" 방법 결 (방법론 마지막 §) 은 본 차의 결정 결로 박지 않음. 별도 차에서 탐구.

10-3. 다음 액션

GO 신호 → M-1 사이클 1 진입 → 5 unit 작업 → 헤드리스 PASS → 사이클 2.

§11. 비교

본 spec 은 다중 맵 - 자료구조 설계.pdf 의 §7 (단계별 롤아웃) 을 방법론 결로 다시 분해한 결. 자료구조 결정은 그 spec, 사이클 / 검증 결은 본 spec.

방법론 자체는 일반 결 (모든 다음 작업에 적용 가능). 본 차는 첫 적용 사례. 후속 차에선 본 spec 결 그대로 다른 작업에 결 적용 가능.